CS 300 – Chapter 10 Notes
Graphs

Jose de Lima

April 21, 2026

Key idea

Why graphs matter. Every previous data structure in this course is really a specialization of a graph: a linked list is a path graph, a tree is a connected acyclic graph, a heap is a tree with an ordering, a hash-chained bucket is a forest. Graphs are the most general model of "structured connections," so they're where interesting algorithms live: shortest paths, scheduling, routing, dependency resolution, network flow, recommendation, social analysis. You'll use them more than any other data structure in the wild.

1 10.1 Introduction

The vocabulary

Definition —

Graph. A pair $G = (V, E)$ where V is a set of *vertices* (nodes) and E is a set of *edges*, each edge connecting two vertices.
Adjacent. Two vertices are adjacent if an edge connects them.
Path. A sequence of edges from a source vertex to a destination vertex. The *length* of a path is the number of edges (or the sum of edge weights, for weighted graphs).
Distance. The length of the *shortest* path between two vertices.
$|V|$ and $|E|$. The number of vertices and edges. By convention, algorithms are analyzed as a function of both; the abbreviations V and E appear inside Big-O notation.

Aside

Watch the word "length" – it's overloaded. An *unweighted* path length is a count of edges; a *weighted* path length is a sum of weights. When someone says "shortest path" without qualifying, they usually mean the weighted version unless context makes it obvious.

The shape of the space

A graph has up to $\binom{V}{2} = \Theta(V^2)$ possible edges (undirected) or $V(V-1) = \Theta(V^2)$ (directed). The real world rarely fills that space:

- **Dense graph**: $|E| = \Theta(V^2)$. Every vertex is connected to a constant fraction of the others. Complete graphs, small road networks within a city block.

- **Sparse graph**: $|E| = O(V)$ or $O(V \log V)$. Each vertex has only a few neighbors. Most real graphs: social networks, road networks, computer networks, the web.

Dense vs. sparse matters *enormously* for algorithm choice. An $O(V^2)$ algorithm on a sparse graph is wasteful; an $O(E \log V)$ algorithm is brilliant on sparse graphs and mediocre on dense ones. The first question to ask about any graph algorithm is: "what does this cost on a sparse input?"

Example

Concrete numbers. Facebook has roughly 3×10^9 users and an average of a few hundred friends per user. That's $V = 3 \times 10^9$, $E \approx 10^{12}$. The dense upper bound would be $V^2 \approx 9 \times 10^{18}$ – seven orders of magnitude larger. An adjacency matrix is physically impossible; an adjacency list is plausible. This is why sparsity assumptions drive the design of real graph systems.

2 10.2 Applications

The textbook section runs through three examples; the interesting claim is the same in each:

Key idea

Graphs model anything with binary relationships. Once your problem is "these things connect to those things," you're working with a graph. The graph abstraction is so ubiquitous that recognizing it in a problem is 90% of the solution.

Navigation (geography)

Cities/intersections are vertices; roads are edges. Edge weights are distances or travel times. Shortest-path queries give driving directions. This is what powers Google Maps, Waze, and every GPS unit made since the 90s.

The interesting twist: edge weights aren't static. Real-time traffic turns the same road into a different edge every few minutes. The right algorithm has to be fast enough that you can *re-solve* every time conditions change, not just once.

Product recommendations (e-commerce)

Products are vertices; "bought together" or "viewed together" is an edge. A customer's basket gives you a starting set; walk one step out along the edges and you have recommendations.

Social and professional networks

People are vertices; relationships are edges. "Friends of friends" is a two-step walk. PageRank, centrality, community detection all live here.

Other common models you'll encounter

- **Compiler dependencies.** Source files are vertices, `#include` edges drive rebuild order – a topological sort of the dependency DAG.
- **Spreadsheet formula dependencies.** Cells are vertices, references are edges. Evaluation order is a topological sort; circular references are a cycle detection problem.

- **Package managers.** Libraries are vertices, dependency constraints are edges. Resolving versions is a constraint problem on the graph.
- **Build systems (`make`, `bazel`, `cmake`).** Targets are vertices, prerequisites are edges. Determining what to rebuild after a change is reachability.
- **State machines.** States are vertices, transitions are edges. Model checking is graph reachability with labels.
- **Game AI and pathfinding.** Tiles or waypoints are vertices, valid moves are edges. A* (shortest path with a heuristic) is the industry standard.
- **Neural networks.** The topology is literally a weighted DAG.

Aside

If you can phrase your problem as "given these things and these connections, what can I reach / what's the best path / what's the order / which connections matter," stop trying to invent algorithms and go find the right graph algorithm instead. That's the whole payoff of the abstraction.

3 10.3 Adjacency Lists

We have the vocabulary. Now: how do we store a graph in memory?

Definition —

Adjacency list. Each vertex carries a list (or vector) of the vertices it's adjacent to. One list per vertex.

C++ representations

Several idioms are common; pick based on your data model.

```
// 1. Vertices as integer IDs 0..V-1, parallel array of neighbor vectors.
//    The fastest and most common choice in competitive programming.
std::vector<std::vector<int>> adj(V);
adj[u].push_back(v);   // add edge u--v
adj[v].push_back(u);   // undirected => add to both lists

// 2. Weighted edges: store (neighbor, weight) pairs.
std::vector<std::vector<std::pair<int,int>>> adj(V);
adj[u].emplace_back(v, weight);

// 3. Vertex objects with owned adjacency data (OOP style).
struct Vertex {
    std::string label;
    std::vector<Vertex*> neighbors;
};
```

Cost analysis

- **Space**: $O(V + E)$. Each vertex's list contributes one header; each edge contributes two list entries (one in each endpoint's list for undirected graphs).

- **"Is u adjacent to v?"**: $O(\deg(u))$. Scan u's list. Worst case $O(V)$ if u is connected to everything.
- **"List all neighbors of u"**: $O(\deg(u))$. Perfect – you just hand over the list.
- **Add/remove edge**: $O(1)$ to add. Removal is $O(\deg(u))$ because you have to find the entry.

Key idea

Adjacency lists are the default choice for sparse graphs. The space win ($V + E$ vs. V^2) is so large that even the slower adjacency query is usually tolerated. Essentially every production graph library (NetworkX, boost::graph, the JDK's graph APIs) uses lists by default.

4 10.4 Adjacency Matrices

Definition —

Adjacency matrix. A $V \times V$ matrix M where $M[i][j] = 1$ if there's an edge from i to j, else 0. For weighted graphs, $M[i][j]$ holds the weight (with ∞ or some sentinel for "no edge").

```
std::vector<std::vector<int>> m(V, std::vector<int>(V, 0));
m[u][v] = 1;
m[v][u] = 1;   // undirected

// Weighted version; INT_MAX = "no edge"
std::vector<std::vector<int>> w(V, std::vector<int>(V, INT_MAX));
w[i][i] = 0;   // zero distance to self
w[u][v] = weight;
```

Cost analysis

- **Space**: $\Theta(V^2)$. Forgiving for small dense graphs, catastrophic for large sparse ones.
- **"Is u adjacent to v?"**: $O(1)$. Just read the matrix cell.
- **"List all neighbors of u"**: $O(V)$. Scan row u even if there are only two neighbors.
- **Add/remove edge**: $O(1)$.
- **Matrix math**: $M^k[i][j]$ is the number of length-k walks from i to j. This is the basis of matrix-multiplication shortest-path algorithms and a ton of algebraic graph theory.

When to pick which

	Adjacency list	Adjacency matrix
Space	$O(V + E)$	$\Theta(V^2)$
Edge query (u, v)?	$O(\deg u)$	$O(1)$
List neighbors of u	$O(\deg u)$	$O(V)$
Iterate all edges	$O(V + E)$	$O(V^2)$
Good for...	sparse, large V	dense, small V

Gotcha

The V^2 space blow-up of an adjacency matrix is lethal well before V seems "large." At $V = 10^5$, a matrix of `int` costs 40 GB; at $V = 10^6$, it's 4 TB. Adjacency lists happily hold the same data in kilobytes or megabytes, depending on edge count. *Always check sparsity before reaching for a matrix.*

Aside

A third representation worth knowing is the **edge list**: just `std::vector<Edge>` with no per-vertex indexing. Useless for most queries, but perfect for Kruskal's MST algorithm (which processes edges in weight order) and for streaming graphs through a network.

5 10.5 Breadth-First Search

Key idea

BFS in one sentence. Explore vertices in order of distance from the start: all distance-1 vertices before any distance-2, all distance-2 before any distance-3, and so on. The queue *is* the algorithm – FIFO ordering is what produces the distance-layered traversal.

Algorithm

```
void bfs(const std::vector<std::vector<int>>& adj, int start) {
    std::vector<bool> discovered(adj.size(), false);
    std::queue<int> frontier;

    frontier.push(start);
    discovered[start] = true;

    while (!frontier.empty()) {
        int u = frontier.front(); frontier.pop();
        visit(u);
        for (int v : adj[u]) {
            if (!discovered[v]) {
                discovered[v] = true;
                frontier.push(v);
            }
        }
    }
}
```

Gotcha

Mark on enqueue, not on dequeue. The `discovered` flag is set *when a vertex is pushed*, not when it's popped. If you mark on pop, the same vertex can be pushed many times through different neighbors, blowing up the queue. The "mark on enqueue" invariant guarantees each vertex is pushed exactly once.

Distance and parent tracking

BFS's real power isn't just "visit everything." It computes *shortest paths in unweighted graphs* for free. Augment each visited vertex with a distance and a parent pointer:

```
void bfs(const std::vector<std::vector<int>>& adj, int start,
         std::vector<int>& dist, std::vector<int>& parent) {
    int n = adj.size();
    dist.assign(n, -1);
    parent.assign(n, -1);
    std::queue<int> q;
    q.push(start);
    dist[start] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
    }
}
```

After BFS, `dist[v]` is the minimum number of edges from `start` to `v`; following `parent` pointers back from `v` reconstructs the path in reverse.

Cost

$O(V + E)$. Each vertex is dequeued once; each edge is scanned at most twice (once from each endpoint, for undirected graphs). Can't be beat for unweighted shortest paths.

Why BFS works – the invariant

Key idea

At any point in BFS, the queue contains vertices at distances $\{d, d, \ldots, d, d+1, d+1, \ldots\}$ – a stretch of one layer, then the next. When a layer-d vertex is popped, its yet-undiscovered neighbors are exactly the layer-$d+1$ vertices reachable through it, and they go to the back of the queue. That's why the FIFO ordering preserves "shortest path"-ness – any alternative path would have been seen (and shorter) on an earlier iteration.

Typical applications

- **Shortest path in unweighted graph** – unbeatable.
- **"Friend of a friend" recommendations** – explore distance-2 and distance-3 neighborhoods.
- **Connected components** – run BFS from every unvisited vertex; each run finds one component.
- **Bipartite check** – 2-color BFS layers; if an edge connects two same-colored vertices, the graph isn't bipartite.

- **Crawling / scraping** – explore a website or peer-to-peer network level-by-level.
- **Closest-of-many search** – as in the peer-to-peer example above, find the first node satisfying a condition (fewest hops).

6 10.6 Depth-First Search

Key idea

DFS in one sentence. Go as far as possible along one path, backtrack only when stuck. A stack (explicit or the call stack) *is* the algorithm.

Iterative DFS

```
void dfs(const std::vector<std::vector<int>>& adj, int start) {
    std::vector<bool> visited(adj.size(), false);
    std::stack<int> s;
    s.push(start);

    while (!s.empty()) {
        int u = s.top(); s.pop();
        if (visited[u]) continue;
        visited[u] = true;
        visit(u);
        for (int v : adj[u])
            if (!visited[v]) s.push(v);
    }
}
```

Gotcha

Iterative DFS marks on pop, not on push. The opposite of BFS. If you mark on push with a stack, the neighbor ordering gets reversed in a subtle way that most people trip over. The idiomatic iterative form above pushes liberally and filters at the top of the loop; duplicate stack entries are OK.

Recursive DFS

Shorter and usually what you want:

```
void dfs(int u, const std::vector<std::vector<int>>& adj,
         std::vector<bool>& visited) {
    if (visited[u]) return;
    visited[u] = true;
    visit(u);
    for (int v : adj[u]) dfs(v, adj, visited);
}
```

Gotcha

Recursion depth. The call stack can be as deep as V on a path-shaped graph. At V around 10^5 or more, on most platforms you'll overflow the stack. Either increase the thread stack limit, or iterate with an explicit stack. This is the single most common bug in graduate-level graph assignments.

Cost

$O(V + E)$, same as BFS – each vertex is visited once, each edge examined at most twice (undirected).

BFS vs. DFS: when to pick which

	BFS	DFS
Auxiliary structure	queue (FIFO)	stack (LIFO)
Shortest path in unweighted graphs	yes	no
Memory (worst case)	$O(V)$ queue	$O(V)$ stack
Explores	breadth-first	depth-first
Natural for	distance, levels	reachability, ordering
Solves	shortest path, connectivity	topological sort, cycle detection, SCCs

Key idea

The deep insight: **DFS produces an ordering** (the order in which vertices are finished), while **BFS produces a layering** (the distance from start). Almost every advanced graph algorithm is some clever use of one or the other – topological sort and SCC from DFS finish times; Dijkstra and shortest paths from BFS-style frontier expansion.

7 10.7 Directed Graphs

Definition —

Directed graph (digraph). Edges have direction. $(u \to v) \in E$ does *not* imply $(v \to u) \in E$.
Cycle. A path that starts and ends at the same vertex.
DAG (directed acyclic graph). A directed graph with no cycles. The single most useful graph class in computer science.

Why DAGs matter

A DAG captures "A must come before B" relationships without contradicting itself. Anything that *orders* things is a DAG:

- Course prerequisites ($\to$ topological sort).
- Build systems ($\to$ incremental rebuild).
- Spreadsheet formulas ($\to$ evaluation order).
- Git commit history ($\to$ merges and ancestors).
- Neural net forward passes.

- Dataflow programs.

If your directed graph has a cycle, you have a contradiction: A needs B first, but B also needs A first. Cycle detection on directed graphs is DFS plus a "currently in recursion" flag.

Representation

Same adjacency list or matrix, but only one direction per edge:

```
// Directed: only add u -> v, not v -> u
adj[u].push_back(v);
```

Terminology that bites

- **In-degree** of v: number of edges ending at v.
- **Out-degree** of v: number of edges starting at v.
- **Successor / predecessor**: v is a successor of u if $(u \to v) \in E$; then u is a predecessor of v.
- **Source**: vertex with in-degree 0 (no one points to it).
- **Sink**: vertex with out-degree 0 (it points to no one).

Example

Kidney exchange as a cycle problem. A directed edge $D \to R$ means "donor family of patient D is willing to give a kidney to recipient R." A k-way simultaneous exchange is a cycle of length k. Hospitals literally run cycle-detection algorithms on these graphs to match transplants – a real-world optimization problem worth billions of dollars in life-years.

8 10.8 Weighted Graphs

Definition —

Weighted graph. Each edge carries a numerical weight (cost, distance, time, capacity, …). Path length = sum of edge weights along the path.

Why weights complicate things

In an unweighted graph, "shortest" means "fewest hops." BFS handles it. In a weighted graph, "shortest" means "smallest total weight" – and a path with more edges can easily be shorter than one with fewer:

$$\mathrm{A} \xrightarrow{10} \mathrm{C} \text{ versus } \mathrm{A} \xrightarrow{2} \mathrm{B} \xrightarrow{3} \mathrm{C} \qquad \text{(length 10 vs. 5)}$$

BFS can't see this. You need an algorithm that explores in order of *weighted distance*, not edge count – enter Dijkstra (§10.9).

Representation

```
// List form: (neighbor, weight) pairs.
std::vector<std::vector<std::pair<int,int>>> adj(V);
```

```
adj[u].emplace_back(v, w);

// Matrix form: INT_MAX (or INFINITY) marks "no edge".
std::vector<std::vector<int>> w(V, std::vector<int>(V, INT_MAX));
w[i][i] = 0;
w[u][v] = weight;
```

Negative weights

Negative edge weights are allowed, but **negative cycles** break the shortest-path problem entirely: you can loop around the negative cycle forever, driving the total cost to $-\infty$. Shortest path is undefined.

Gotcha

Dijkstra can't handle negative weights, period. Its correctness proof relies on never having to revisit a "finished" vertex, and negative edges can make that false. Use Bellman-Ford (§10.10) or the special Johnson's algorithm if any edge is negative. Knowing which shortest-path algorithm tolerates which edge signs is a classic exam question.

9 10.9 Dijkstra's Shortest Path

Key idea

Dijkstra in one sentence. BFS, but the frontier is a priority queue keyed by distance. Always expand the closest unvisited vertex next; its distance is now final.

Algorithm

For every vertex v, maintain:

- `dist[v]`: best-known shortest distance from the source (initially ∞).
- `prev[v]`: previous vertex along that best-known path (initially null).

At each step, pull the vertex with smallest tentative `dist` out of the priority queue (it won't improve), and *relax* all its outgoing edges: for each edge (u, v, w), if `dist[u] + w < dist[v]`, update `dist[v]` and `prev[v]`.

```
#include <queue>
#include <vector>
#include <limits>

using Edge = std::pair<int,int>;   // (neighbor, weight)
const int INF = std::numeric_limits<int>::max();

void dijkstra(const std::vector<std::vector<Edge>>& adj, int src,
              std::vector<int>& dist, std::vector<int>& prev) {
    int n = adj.size();
    dist.assign(n, INF);
    prev.assign(n, -1);
    dist[src] = 0;
```

```
    // Min-heap of (distance, vertex). std::greater flips the default max-heap
        .
    using PQE = std::pair<int,int>;
    std::priority_queue<PQE, std::vector<PQE>, std::greater<PQE>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;          // stale entry -- already improved
        for (auto [v, w] : adj[u]) {
            int alt = d + w;
            if (alt < dist[v]) {
                dist[v] = alt;
                prev[v] = u;
                pq.push({alt, v});
            }
        }
    }
}
```

Gotcha

The "stale entry" trick. `std::priority_queue` doesn't support decrease-key, so whenever we improve a vertex's distance, we just push a new, smaller entry. When we later pop, we check `if (d > dist[u]) continue;` to skip outdated entries. This is the idiomatic C++ Dijkstra – simple, fast, and what you'd write in interviews or contests.

Why it works – greedy correctness

Key idea

When we pop the closest remaining vertex u with tentative distance d, *no later path can possibly be shorter*. Any other path to u goes through some unvisited vertex v with $\texttt{dist}[v] \geq d$ (that's why u was chosen first), and then adds a non-negative edge weight on top of $\texttt{dist}[v]$, giving total $\geq d$. So d is final at pop time. This is exactly why **Dijkstra requires non-negative edges**: the argument breaks if a negative edge can shave below d after the fact.

Cost

Priority-queue implementation	Runtime
Unsorted array / linear scan	$O(V^2 + E)$
Binary heap (`std::priority_queue`)	$O((V + E) \log V)$
Fibonacci heap	$O(E + V \log V)$

For sparse graphs, the binary-heap version is the sweet spot: simple, fast, what you'd reach for in real code. Fibonacci heaps have better theoretical bounds but rarely win in practice because of their hideous constant factors.

Reconstructing the path

After Dijkstra, walk prev backwards from the destination:

```
std::vector<int> pathTo(int target, const std::vector<int>& prev) {
    std::vector<int> path;
    for (int v = target; v != -1; v = prev[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}
```

If `prev[target] == -1` and `target != src`, the target is unreachable.

10 10.10 Bellman-Ford Shortest Path

Key idea

Bellman-Ford in one sentence. Relax every edge $V - 1$ times. If any edge can *still* be relaxed after that, there's a negative cycle. Slower than Dijkstra but handles negative weights.

Algorithm

```
struct Edge { int u, v, w; };

bool bellmanFord(int n, const std::vector<Edge>& edges, int src,
                 std::vector<int>& dist, std::vector<int>& prev) {
    dist.assign(n, INF);
    prev.assign(n, -1);
    dist[src] = 0;

    // Main loop: relax every edge V-1 times.
    for (int i = 0; i < n - 1; ++i) {
        for (const auto& e : edges) {
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
                prev[e.v] = e.u;
            }
        }
    }

    // Negative-cycle check: one more pass -- if anything still relaxes,
    // the graph has a reachable negative cycle.
    for (const auto& e : edges) {
        if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v])
            return false;
    }
    return true;
}
```

Why $V - 1$ iterations?

A shortest path has at most $V - 1$ edges (any more would revisit a vertex, wasting weight if weights are non-negative – and if weights are negative, see below). Each iteration propagates "correct" distances one edge farther along. After $V - 1$ passes, every shortest path has been fully propagated, as long as no negative cycles exist.

If an edge is *still* relaxable on a V-th iteration, then some path keeps getting shorter – a negative cycle is the only way that can happen.

Cost

$O(VE)$. Roughly V times slower than Dijkstra in the worst case, but:

- Handles negative weights.
- Detects negative cycles.
- Works on *any* graph representation – just give it a list of edges.
- Easy to parallelize (independent edge relaxations).

Dijkstra vs. Bellman-Ford, decision flowchart

Situation	Use
All edges non-negative, sparse	Dijkstra + binary heap
Negative edges, no negative cycles	Bellman-Ford
Need to detect negative cycles	Bellman-Ford
All-pairs shortest paths	Floyd-Warshall (§10.13)
Unweighted graph	BFS (not either of these)

Aside

SPFA ("Shortest Path Faster Algorithm") is a queue-based refinement of Bellman-Ford that's often faster in practice – only re-process vertices whose distance actually changed. Worst case is still $O(VE)$, but average behavior on real graphs is much better.

11 10.11 Topological Sort

Definition —

Topological sort. A linear ordering of a DAG's vertices such that every edge $u \to v$ has u appearing before v. Exists iff the graph is acyclic. Not unique: most DAGs have many valid orderings.

Key idea

The canonical use case. Whenever you have "do A before B" constraints and want a valid linear schedule, you want topological sort. Course prerequisites, build orders, spreadsheet evaluation, task scheduling, dataflow execution – all the same problem.

Kahn's algorithm (in-degree method)

Repeatedly peel off vertices with no incoming edges – they're safe to do first.

```
std::vector<int> topoSortKahn(int n,
                              const std::vector<std::vector<int>>& adj) {
    std::vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) ++indeg[v];

    std::queue<int> q;
    for (int u = 0; u < n; ++u)
        if (indeg[u] == 0) q.push(u);

    std::vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);
    }

    if ((int)order.size() != n) return {};  // cycle detected
    return order;
}
```

If the final order doesn't have all n vertices, the remaining vertices form a cycle – automatic cycle detection, no extra work.

DFS-based topological sort

Alternative: DFS, and *push each vertex onto a result stack when it finishes* (after its recursion returns). Reverse the stack.

```
void dfsTopo(int u, const std::vector<std::vector<int>>& adj,
             std::vector<bool>& visited, std::vector<int>& out) {
    visited[u] = true;
    for (int v : adj[u])
        if (!visited[v]) dfsTopo(v, adj, visited, out);
    out.push_back(u);   // post-order: u is "done" now
}

std::vector<int> topoSortDfs(int n,
                             const std::vector<std::vector<int>>& adj) {
    std::vector<bool> visited(n, false);
    std::vector<int> out;
    for (int u = 0; u < n; ++u)
        if (!visited[u]) dfsTopo(u, adj, visited, out);
    std::reverse(out.begin(), out.end());
    return out;
}
```

Why the reversal? DFS finishes deeper-in-the-dependency-chain vertices first. In a topological ordering,

those should come *last* (they depend on earlier stuff). Reversing puts them in the right order.

Cost

Both are $O(V + E)$.

Kahn vs. DFS

- **Kahn** is iterative and natural for streaming/parallel processing – multiple vertices can be in the "ready" queue simultaneously.
- **DFS** is shorter to write and fits naturally with other DFS-based algorithms (SCC, cycle detection).

Pick based on context. Most textbooks teach DFS; most real-world schedulers (like Tarjan's, or `make`'s job dispatcher) use Kahn-style because they can work on "what's ready now" incrementally.

12 10.12 Minimum Spanning Tree

Definition —

Spanning tree. A subset of a connected graph's edges that connects all vertices and forms a tree ($V - 1$ edges, no cycles).
Minimum spanning tree (MST). A spanning tree with the smallest possible sum of edge weights. Requires a connected, undirected, weighted graph.

Example

Physical intuition. You're the electric company. Each edge is a potential power line between two cities, with a cost (length, terrain difficulty). You must connect every city. What's the cheapest wiring plan? That's an MST.

Kruskal's algorithm (greedy edges)

Sort all edges by weight. Add each edge to the MST unless it would create a cycle. Stop when you have $V - 1$ edges.

"Would it create a cycle?" is answered with a **union-find** (disjoint-set) data structure: two vertices are in the same component iff their roots are equal.

```
struct DSU {
    std::vector<int> parent, rank_;
    DSU(int n) : parent(n), rank_(n, 0) {
        std::iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];   // path compression
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {
```

```
        a = find(a); b = find(b);
        if (a == b) return false;
        if (rank_[a] < rank_[b]) std::swap(a, b);
        parent[b] = a;
        if (rank_[a] == rank_[b]) ++rank_[a];
        return true;
    }
};

struct Edge { int u, v, w; };

std::vector<Edge> kruskal(int n, std::vector<Edge> edges) {
    std::sort(edges.begin(), edges.end(),
              [](const Edge& a, const Edge& b){ return a.w < b.w; });

    DSU dsu(n);
    std::vector<Edge> mst;
    for (const auto& e : edges) {
        if (dsu.unite(e.u, e.v)) {
            mst.push_back(e);
            if ((int)mst.size() == n - 1) break;
        }
    }
    return mst;
}
```

Cost

$O(E \log E) = O(E \log V)$ (since $E \leq V^2$, the two are equivalent up to a constant factor). Dominated by the sort; union-find operations are near-$O(1)$ amortized.

Why the greedy works

Key idea

Cut property. For any "cut" (partition of vertices into two sets), the minimum-weight edge crossing the cut is in *some* MST. Kruskal's always adds the next-smallest edge that connects two different components – that's exactly a minimum edge across the cut between those components. So each added edge is safe.

Prim's algorithm (greedy vertex frontier)

Alternative MST algorithm: start from any vertex, repeatedly add the cheapest edge that connects an "inside" vertex to an "outside" vertex. Structurally it's Dijkstra with a different relaxation rule (compare edge weight directly, not cumulative distance).

	Kruskal	Prim
Approach	greedy on edges	greedy on vertex frontier
Needs	sorted edge list, union-find	adjacency list, priority queue
Cost	$O(E \log E)$	$O(E \log V)$ w/ binary heap
Good when	edges are easy to enumerate	graph is dense, adjacency is fast

Aside

For a dense graph ($E = \Theta(V^2)$), Prim with an array-based priority queue runs in $O(V^2)$, which beats Kruskal's $O(V^2 \log V)$. For sparse graphs, they're essentially tied.

13 10.13 All-Pairs Shortest Path (Floyd-Warshall)

Key idea

Floyd-Warshall in one sentence. Compute shortest path from *every* vertex to every other vertex by, in turn, allowing each vertex as an intermediate step. Three nested loops, no priority queue, $O(V^3)$.

Algorithm

Maintain `dist[i][j]` = current best-known distance from i to j. Initialize:

- `dist[i][i] = 0`.
- `dist[i][j] = w(i,j)` if the edge exists.
- `dist[i][j] =` ∞ otherwise.

Then iterate: for each intermediate vertex k, for each pair (i, j), ask "does going through k beat the current best?"

```
void floydWarshall(std::vector<std::vector<int>>& dist) {
    int n = dist.size();
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (dist[i][k] != INF && dist[k][j] != INF
                    && dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
}
```

Why it works – the k-loop invariant

Key idea

After the k-th outer iteration, `dist[i][j]` is the shortest path from i to j using *only vertices* $\{0, 1, \ldots, k\}$ *as intermediates*. When $k = n - 1$, that's the shortest path using any vertex, which is the answer. The k loop must be *outermost* – swapping the loop order breaks the invariant and gives wrong answers. This is a favorite homework trap.

Cost

$O(V^3)$ time, $O(V^2)$ space. That's cubic, but with tiny constants – three nested for loops of integer additions. On modern hardware it's vectorizable and absolutely flies for V up to a few thousand.

Floyd-Warshall vs. running Dijkstra V times

	Dijkstra from every source	Floyd-Warshall
Total cost (binary heap)	$O(V(V+E)\log V)$	$O(V^3)$
Handles negative edges?	no	yes
Detects negative cycles?	no	yes (check diagonal)
Code complexity	moderate	very simple
Ideal on	sparse, non-negative	dense, or need negative-weight support

Gotcha

Negative cycle detection. After Floyd-Warshall, if any `dist[i][i] < 0`, there's a negative cycle through vertex i. Slick – no extra pass needed.

Path reconstruction

Floyd-Warshall only computes distances. To recover paths, track the intermediate vertex alongside:

```
std::vector<std::vector<int>> next;   // next[i][j] = first hop from i toward
    j

// During init: next[i][j] = j if edge exists, else -1; next[i][i] = i.
// During update:
if (dist[i][k] + dist[k][j] < dist[i][j]) {
    dist[i][j] = dist[i][k] + dist[k][j];
    next[i][j] = next[i][k];
}

// Reconstruct path i -> j:
std::vector<int> path = {i};
while (path.back() != j) path.push_back(next[path.back()][j]);
```

Algebraic view

Floyd-Warshall is min-plus matrix multiplication in disguise. Replace "product + sum" with "sum + min," and the adjacency matrix raised to high enough a power (in this semiring) is exactly the all-pairs-shortest-path matrix. This connects shortest paths to a whole field of tropical algebra, used in everything from compiler optimization to network calculus.

Key idea

Chapter takeaway. Graphs are the universal data structure of computer science. Every major algorithm in this chapter – BFS, DFS, Dijkstra, Bellman-Ford, topological sort, Kruskal, Floyd-Warshall – is an instance of "explore the graph by expanding a frontier in some specific order." The frontier type (queue, stack, priority queue, edge-sorted list, triple loop) and the relaxation rule (visit once, update-if-shorter, etc.) are what distinguish them. Internalize the pattern and you can reason about a new graph algorithm in minutes.